



المدرسة الوطنية للذكاء الاصطناعي والرقمنة - بركان  
ÉCOLE NATIONALE DE L'INTELLIGENCE ARTIFICIELLE ET DU DIGITAL - BERKANE  
ИНСАНОО. ИНТЕЛ. ИННАК. ИНТЕЛОНХ ПОНОВЦИО - БОРКО

# ADAS Test Case Generation

using agentic AI

*Un système multi-agents pour la génération, l'évaluation et l'amélioration itérative automatisées des cas de tests fonctionnels ADAS à partir des exigences et des vidéos de conduite*

## Rapport de Projet de Fin d'Études

**École de diplôme :** École nationale de l'intelligence artificielle et digitale berkane

**Spécialité de diplôme :** Ingénieur en intelligence artificielle

**Référent :** Université Mohamed premier

**Année :** 2025/2026

Stage realise au sein de Capgemini Engineering.

Réalisé par :

**OUKACHA Ahmed**

## Encadré par :

Membres du jury avec voix délibérative

|             |        |
|-------------|--------|
| <b>name</b> | status |
| title       |        |
| <b>name</b> | status |
| title       |        |
| <b>name</b> | status |
| title       |        |
| <b>name</b> | status |
| title       |        |

**Au** terme de ce projet de fin d'études, nous tenons à exprimer notre reconnaissance à celles et ceux qui ont jalonné notre parcours et contribué à la réussite de ce travail. Nos premiers remerciements s'adressent à Capgemini Engineering. Nous y avons trouvé bien plus qu'un lieu de stage : une véritable immersion dans un projet innovant et au cœur des technologies de pointe, alliant systèmes **ADAS** et intelligence artificielle. Nous tenons à remercier chaleureusement nos encadrants pour leur accompagnement précieux. Leur disponibilité constante, leurs conseils avisés et la confiance qu'ils nous ont accordée ont été de véritables moteurs tout au long de cette expérience. Leur expertise technique et leur soutien ont été déterminants. Travailler aux côtés des équipes de **Capgemini Engineering** a été un immense plaisir. Au-delà des tâches quotidiennes, ce sont les échanges partagés, les retours d'expérience et l'esprit de solidarité au sein de l'équipe qui ont enrichi notre vision professionnelle. Nous tenons également à saluer **l'École Nationale d'Intelligence Artificielle et du Digital de Berkane (ENIAD)**. Merci à l'ensemble du corps professoral pour la rigueur de la formation et la qualité des connaissances transmises, qui constituent aujourd'hui le socle de nos compétences. Enfin, que toutes les personnes qui ont soutenu ce projet, de près ou de loin, trouvent ici l'expression de notre gratitude pour avoir rendu cette aventure humaine et professionnelle aussi mémorable.

L'émergence des systèmes avancés d'aide à la conduite (**ADAS**) pose un défi majeur : la complexité croissante des processus de vérification et d'audit. L'analyse des exigences, la conception des plans de test et la création des scénarios de test sont des tâches essentielles mais chronophages en ingénierie. C'est précisément là qu'intervient ce projet de fin d'études. Quel est l'objectif ? Concevoir une plateforme intelligente, basée sur une approche de **agentic-IA**, pour assister les ingénieurs ADAS. Notre solution coordonne le travail de plusieurs agents spécialisés qui collaborent pour analyser les spécifications, structurer les données et produire des cas de test cohérents et traçables. En arrière-plan, l'architecture exploite la puissance des grands modèles de langage (LLM) et utilise le framework LangGraph pour coordonner le flux entre les agents. Plutôt que d'automatiser aveuglément l'ensemble du processus, nous avons intégré une approche d'« intervention humaine » afin que l'expert reste le décideur final. De plus, la mémoire des agents permet au système de tirer profit des connaissances acquises lors des opérations. Techniquement, la solution est fournie via les services FastAPI et déployée dans des conteneurs Docker. Pour surveiller l'intégrité et les performances du système, nous avons implémenté des mécanismes de supervision à l'aide de Grafana. Les résultats confirment la validité de cette approche : combiner l'intelligence artificielle à la supervision humaine améliore la productivité tout en garantissant des résultats de tests standardisés et de haute qualité.

### MOTS-CLÉS

Concepts principaux du projet et accès rapide à la liste des acronymes.

Agentic AI

ADAS

Large Language Models

LangGraph

Human-in-the-Loop

Agent Memory

Test Plan Generation

Test Case Generation

FastAPI

Docker

# LISTE DES ACRONYMES

| Acronyme | Signification                                                                                 |
|----------|-----------------------------------------------------------------------------------------------|
| ACC      | Adaptive Cruise Control — Régulateur de vitesse adaptatif                                     |
| ADAS     | Advanced Driver Assistance Systems — Systèmes avancés d'aide à la conduite                    |
| AEB      | Automatic Emergency Braking — Freinage d'urgence automatique                                  |
| API      | Application Programming Interface — Interface de programmation applicative                    |
| BSW      | Blind Spot Warning — Avertissement d'angle mort                                               |
| CDC      | Cahier des charges — Document de spécification du projet                                      |
| CI/CD    | Continuous Integration / Continuous Deployment — Intégration et déploiement continus          |
| CORS     | Cross-Origin Resource Sharing — Partage de ressources entre origines                          |
| ESC      | Electronic Stability Control — Contrôle électronique de stabilité                             |
| FCW      | Forward Collision Warning — Avertissement de collision frontale                               |
| GenAI    | Generative Artificial Intelligence — Intelligence artificielle générative                     |
| HMI      | Human-Machine Interface — Interface homme-machine                                             |
| ISO      | International Organization for Standardization — Organisation internationale de normalisation |
| JWT      | JSON Web Token — Jeton d'authentification web                                                 |
| LKA      | Lane Keeping Assist — Aide au maintien de voie                                                |
| LLM      | Large Language Model — Grand modèle de langage                                                |
| MCP      | Model Context Protocol — Protocole de contexte pour modèles                                   |

| Acronyme | Signification                                                           |
|----------|-------------------------------------------------------------------------|
| MVP      | Minimum Viable Product — Produit minimum viable                         |
| R2T      | Requirements to Tests — Des exigences aux tests                         |
| RAG      | Retrieval-Augmented Generation — Génération augmentée par récupération  |
| TC       | Test Case — Cas de test                                                 |
| TTC      | Time To Collision — Temps avant collision                               |
| TSR      | Traffic Sign Recognition — Reconnaissance des panneaux de signalisation |
| VS       | Video Scenario — Scénario extrait de la vidéo de conduite               |

|                                        |           |
|----------------------------------------|-----------|
| <b>1 Introduction générale</b>         | <b>9</b>  |
| 1.1 Contexte général                   | 9         |
| 1.2 Problématique                      | 9         |
| 1.3 Objectifs du projet                | 10        |
| 1.4 Choix de l'approche : IA Agentique | 11        |
| 1.5 Organisation du mémoire            | 11        |
| <b>2 État de l'art</b>                 | <b>13</b> |
| <b>3 Architecture du système</b>       | <b>14</b> |
| <b>4 Implementation</b>                | <b>15</b> |
| <b>5 Évaluation</b>                    | <b>16</b> |

# LISTE DES TABLEAUX

---

Texte provisoire.

Texte provisoire.

# Introduction générale

Contexte, problématique, objectifs et organisation du mémoire

---

## 1.1 Contexte général

L'industrie automobile traverse une transformation profonde portée par l'intégration croissante de systèmes intelligents dans les véhicules. Parmi ces avancées, les systèmes avancés d'aide à la conduite (ADAS) occupent une place centrale. Ces systèmes — régulateur de vitesse adaptatif (ACC), freinage d'urgence automatique (AEB), aide au maintien de voie (LKA) — contribuent à la réduction des accidents et à l'amélioration du confort de conduite. Leur complexité fonctionnelle ne cesse cependant de croître, imposant des processus de validation de plus en plus rigoureux.

Dans ce contexte, la vérification des exigences fonctionnelles ADAS représente un enjeu stratégique pour les équipementiers et constructeurs automobiles. Chaque fonction ADAS est définie par un ensemble d'exigences formelles qui décrivent les conditions d'activation, les seuils de déclenchement, les comportements attendus, les contraintes temporelles et les messages conducteur. À partir de ces exigences, les équipes de validation doivent concevoir des cas de test exhaustifs couvrant les scénarios nominaux, les cas limites, les situations d'erreur et les scénarios rares.

Ce stage de fin d'études s'inscrit dans cette problématique et a été réalisé au sein de **Capgemini Engineering**, acteur majeur du conseil et de l'ingénierie dans le secteur automobile.

## 1.2 Problématique

Le processus actuel de génération des cas de test ADAS présente plusieurs limitations structurelles qui motivent cette recherche :

**Génération manuelle.** Les cas de test sont aujourd'hui rédigés manuellement par les ingénieurs de validation, à partir de leur lecture et interprétation des exigences fonctionnelles. Ce processus est chronophage, sujet aux erreurs d'interprétation et difficilement reproductible.

**Couverture incomplète.** La conception manuelle tend à privilégier les scénarios nominaux au détriment des cas limites (*boundary*), des scénarios négatifs et des situations

rares, ce sont précisément ces cas qui révèlent les défauts les plus critiques dans les systèmes de sécurité.

**Absence de traçabilité structurée.** Le lien entre une exigence source et les cas de test qui la couvrent n'est pas toujours formalisé, rendant difficile l'analyse de couverture et la maintenance des suites de tests.

**Temps de cycle élevé.** Le cycle complet — de la réception des exigences à la livraison des cas de test validés — peut s'étendre sur plusieurs semaines, retardant la mise sur le marché des fonctions ADAS.

**Pas d'exploitation des données de conduite.** Les vidéos de conduite réelle, riches en scénarios de terrain, ne sont pas exploitées pour enrichir la conception des tests.

La problématique centrale de ce travail peut ainsi se formuler :

### Problématique

Comment automatiser la génération de cas de test ADAS à partir d'exigences fonctionnelles en langage naturel, tout en garantissant une couverture complète, une traçabilité rigoureuse et une qualité conforme aux standards de l'industrie automobile ?

## 1.3 Objectifs du projet

Pour répondre à cette problématique, le projet ADAS-R2T (*Requirements to Tests*) poursuit les objectifs suivants :

1. **Automatiser la chaîne complète** de transformation des exigences en cas de test : de l'ingestion du fichier Excel source jusqu'à la génération du fichier Excel de sortie contenant les cas de test structurés.
2. **Garantir une couverture exhaustive** en analysant chaque exigence selon cinq dimensions complémentaires (transitions d'état, contraintes temporelles, messages HMI, calculs, cas génériques) et en générant systématiquement quatre catégories de tests : nominal, boundary, negative et rare.
3. **Assurer la traçabilité** entre chaque cas de test généré et l'exigence source, avec un identifiant unique et un lien explicite.
4. **Intégrer l'analyse vidéo** pour enrichir les cas de test avec des scénarios extraits de vidéos de conduite réelle, apportant un réalisme que la seule lecture des exigences ne peut fournir.

5. **Concevoir un système vendor-agnostic** capable de fonctionner avec différents fournisseurs de modèles de langage (OpenAI, Ollama, Gemini) sans modification du code.

## 1.4 Choix de l'approche : IA Agentique

Une question fondamentale s'est posée dès le début du projet : faut-il utiliser l'IA Générative (GenAI) classique ou l'IA Agentique pour résoudre ce problème ?

Pour trancher, nous avons appliqué une **matrice de décision à cinq questions** issue de la littérature :

| Critère            | Analyse du projet                                                                                                               | Résultat  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Sortie</b>      | Le projet demande un résultat exécuté (fichier Excel structuré, KPIs de couverture), pas seulement du texte généré.             | Agentique |
| <b>Trajectoire</b> | Le flux est dynamique et cyclique : générer → évaluer → corriger → régénérer. Les étapes ne sont pas linéaires.                 | Agentique |
| <b>Erreur</b>      | Le système doit s'auto-corriger : si la couverture est insuffisante, il relance la génération ciblée sans intervention humaine. | Agentique |
| <b>Interaction</b> | L'humain définit l'objectif (les exigences) et le système conduit le processus complet de bout en bout.                         | Agentique |
| <b>État</b>        | Le système modifie l'environnement : il lit des fichiers, exécute des analyses, génère des fichiers de sortie.                  | Agentique |

Tableau 1. – Application de la matrice de décision GenAI vs Agentique au projet ADAS-R2T.

Le résultat est unanime : **5 critères sur 5** orientent vers l'IA Agentique. Le système ne se contente pas de générer du contenu — il planifie, exécute, évalue et corrige de manière autonome.

## 1.5 Organisation du mémoire

Ce mémoire est structuré en cinq chapitres :

**Chapitre 2 — État de l'art.** Présente les fondements théoriques nécessaires : l'évolution de l'IA générative vers l'IA agentique, les patrons architecturaux des systèmes multi-

agents, les frameworks d'orchestration (LangGraph) et les travaux existants en génération automatique de tests.

**Chapitre 3 — Architecture du système.** Détaille l'architecture complète du pipeline ADAS-R2T : les 18 nœuds, les quatre stages, les mécanismes de parallélisation, le routage conditionnel et la boucle d'évaluation-correction.

**Chapitre 4 — Implémentation.** Décrit les choix techniques, la couche LLM vendor-agnostic, le framework مِصْبَاح pour l'ingénierie des prompts, l'intégration de l'analyse vidéo et le déploiement via FastAPI.

**Chapitre 5 — Évaluation.** Présente les résultats quantitatifs et qualitatifs, l'évolution des métriques de qualité au fil des itérations et les limites identifiées.

**Conclusion.** Synthétise les contributions, discute les limitations et propose des perspectives d'évolution.

# État de l'art

Contexte, objectifs et organisation du mémoire

---

# Architecture du système

Contexte, objectifs et organisation du mémoire

---

# Implementation

Contexte, objectifs et organisation du mémoire

---

# Évaluation

Contexte, objectifs et organisation du mémoire

---

- [1] Anthropic. *Building effective agents*. Blog technique, 2024. Disponible sur : <https://www.anthropic.com/research/building-effective-agents>. Consulté le 25/03/2026.
- [2] LangChain Inc. *LangGraph Documentation*. 2024-2026. Disponible sur : <https://langchain-ai.github.io/langgraph/>. Consulté le 20/03/2026.
- [3] OpenAI. *GPT-4o Technical Report*. 2024. Disponible sur : <https://openai.com/index/gpt-4o-system-card/>. Consulté le 15/03/2026.
- [4] ISO 26262:2018. *Road vehicles — Functional safety*. International Organization for Standardization, 2018.
- [5] S. Yao, J. Zhao, D. Yu et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2022.
- [6] P. Lewis, E. Perez, A. Piktus et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
- [7] J. Wei, X. Wang, D. Schuurmans et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS, 2022.
- [8] T. B. Brown, B. Mann, N. Ryder et al. *Language Models are Few-Shot Learners*. NeurIPS, 2020. (GPT-3)
- [9] A. Vaswani, N. Shazeer, N. Parmar et al. *Attention Is All You Need*. NeurIPS, 2017. (Architecture Transformer)
- [10] S. Timofte, S. Hendel. *Automated Test Case Generation using Large Language Models: A Survey*. arXiv, 2024.
- [11] European Commission. *Regulation (EU) 2019/2144 — General Safety Regulation for motor vehicles*. Official Journal of the European Union, 2019.
- [12] SAE International. *SAE J3016 — Taxonomy and Definitions for Terms Related to Driving Automation Systems*. 2021.
- [13] FastAPI Documentation. *FastAPI — Modern web APIs with Python*. Disponible sur : <https://fastapi.tiangolo.com/>. Consulté le 10/03/2026.
- [14] Pydantic Documentation. *Pydantic v2 — Data validation using Python type annotations*. Disponible sur : <https://docs.pydantic.dev/>. Consulté le 10/03/2026.

Texte provisoire.